Composable Spells (A Functional Pearl)

YAO LI, Portland State University, USA

Spell systems in tabletop and computer role-playing games are notoriously non-compositional: to cast a frost bolt, a healing aura, or a lightning cone, a player must find each as a pre-defined entry in a spell list rather than assembling it from reusable components. We present a compositional spell system inspired by functional programming. A spell is built in two phases: first a targeting profile is assembled from orthogonal components (allegiance, geometry, count, and origin), then the target is paired with an effect and a duration. A type system with a small number of compatibility conditions ensures that every well-typed spell has a well-defined denotational semantics—it "cannot go wrong". We prove a semantic well-formedness theorem and discuss how the system applies to D&D 5e, reinterpreting spell slots, metamagic, concentration, and upcasting as operations on spell components.

Disclaimer

This paper is the result of experimenting paper writing with AI (Claude Code). All its findings and results should not be taken without a critical inspection.

1 Introduction

You are playing a wizard in a tabletop role-playing game. You want to shoot a bolt of frost at a single enemy—not fire, not lightning, frost—and you want it to travel across the room rather than touch your fingertips. You scan the spell list. There is *Ray of Frost*. It is ranged, it deals cold damage, and it is instant. That is exactly what you wanted.

Now you want to slow three enemies clustered together with a cone of frost. Back to the spell list. There is *Cone of Cold*, but it deals cold damage to everything in the cone, friend or foe. There is no "frost cone, enemies only". You settle for Cone of Cold and hope your allies step aside.

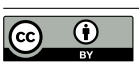
Now you want a healing mist that restores a small amount of health to all allies within ten feet of you, every round, for one minute. There is no such spell. The closest candidates are *Aura of Vitality*, which targets one ally per round, and *Mass Cure Wounds*, which is a single burst. You want the intersection of their features, but the spell list is a catalogue, not an algebra.

This is the fundamental problem: traditional spell systems are *enumerative*. Every combination of effect, delivery, target filter, and duration must be anticipated by the designer and entered as a separate entry. The combinatorial explosion is enormous—and most combinations are simply absent.

The functional programming tradition offers a different philosophy: build large things by composing small, orthogonal pieces. Parser combinators build parsers from characters and concatenation. Functional reactive programming builds interactive programs from signals and sampling. In each case, a small set of primitives and a handful of composition laws yield a space of programs that is far richer than any enumeration could provide.

We apply this philosophy to spell design. A spell in our system is not looked up—it is composed. One chooses an *effect* (*frost*, *fire*, *heal*, …), a *targeting profile* (who is affected: self, allies, enemies; in what area: a point, a cone, a splash; and how many), and a *duration* (instant, timed, persistent). The type system checks that the combination is coherent. Every well-typed spell has a well-defined denotational semantics: it "cannot go wrong".

Author's Contact Information: Yao Li, Portland State University, USA, liyao@pdx.edu.



Two senses of composition. It is worth being precise about what kind of composition we study. We are concerned with *component composition*: assembling a *single* spell from its constituent parts. This is distinct from *spell-to-spell composition*: applying multiple spells in sequence or allowing one spell's effect to interact with another's. The latter—fire melting ice, electricity conducting through water—is interesting in its own right, and games like *Divinity: Original Sin 2* explore it extensively. We leave spell-to-spell composition aside; it is component composition that current systems handle poorly, and component composition that we address here.

The pearl. We present the system as a functional pearl. The contributions are not a novel algorithm or a new proof technique, but a clean design and the insight that a familiar body of theory—product types, partial orders, denotational semantics—fits a problem that game designers have struggled with informally for decades.

The paper proceeds as follows. Section 2 introduces the system informally and contrasts it with two existing game systems: Tyranny's sigil-based spell crafting and Ars Magica's Verb–Noun magic. Section 3 defines System S, the single-effect spell language, including its syntax and two-phase composition operators. Section 4 gives the type system and denotational semantics, and proves the semantic well-formedness theorem. Section 5 extends to System M, where a spell may carry a set of simultaneously applied effects. Section 6 applies the framework to D&D 5e. Section 7 discusses related work, and Section 8 concludes.

2 A Spell by Any Other Name

Before formalising anything, let us walk through the system informally. We will build a frost bolt step by step, contrast the result with what existing systems offer, and sketch the key structure of the target type that will be made precise in Section 3.

2.1 Building a Spell from Components

A spell in our system has three top-level components: a *targeting profile*, an *effect*, and a *duration*. We compose them using two operators. Within a phase, $\otimes$ combines independent components; across phases, $\langle \cdot, \cdot \rangle$ pairs the targeting profile with the effect and duration.

The targeting profile itself is assembled from four sub-components:

- **Origin**: where the spell originates—from the caster (*Caster*) or from a chosen point in the world (*Point(p)*).
- **Allegiance**: which entities are eligible targets—Self (the caster only), Ally (the caster and allies), Enemy, or Any.
- **Geometry**: the shape of the affected area—a Point, a Line, a Cone, a Splash of radius r, or the special form GAll (all eligible entities in range).
- **Count**: how many of the eligible entities within the area are affected—$N(n)$ for a fixed number $n \geq 1$, or CAll.

To build a frost bolt aimed at a single enemy across the room, one writes:

$$t_{\text{bolt}} = \mathsf{origin}(\mathit{Point}(p)) \otimes \mathsf{alleg}(\mathsf{Enemy}) \otimes \mathsf{geom}(\mathsf{Point}) \otimes \mathsf{count}(N(1))$$

$$s_{\text{frostbolt}} = \langle t_{\text{bolt}},\ \mathsf{eff}(\mathit{Frost}) \otimes \mathsf{dur}(\mathsf{Instant}) \rangle$$

That is it. No spell list, no memorisation slot, no hunting for "Ray of Frost" in an index. The frost bolt is built directly from the components that describe it.

Now the healing aura from the introduction—all allies within a five-metre splash, every round, for one minute:

$$t_{\text{aura}} = \mathsf{origin}(\mathit{Caster}) \otimes \mathsf{alleg}(\mathsf{Ally}) \otimes \mathsf{geom}(\mathsf{Splash}(5\mathrm{m})) \otimes \mathsf{count}(\mathsf{CAll})$$

$$s_{\text{aura}} = \langle t_{\text{aura}},\ \mathsf{eff}(\mathit{Heal}) \otimes \mathsf{dur}(\mathsf{Timed}(1\text{min}))\rangle$$

Both spells are composed from the same vocabulary of components. The difference lies entirely in the component values chosen. The type system—described in Section 4—rejects combinations that are incoherent, such as healing one's enemies.

2.2 What Goes Wrong in Existing Systems

Tyranny. The closest existing system is the sigil-based spell crafting in *Tyranny* (Obsidian Entertainment, 2016) [13]. Players combine a *Core Sigil* (one of eleven elemental types: Fire, Frost, Lightning, Life, *etc.*) with an *Expression Sigil* (one of nine delivery forms: bolt, cone, aura, *etc.*) and optionally stack *Accent Sigils* (numerical tweaks to range, damage, or duration).

On the surface this resembles our system. The resemblance is superficial. Tyranny's system is not compositional—it is a *catalogue* of pre-made spells (roughly sixty-four, based on community documentation [1]) that are gradually unlocked as the player acquires sigils. Not all Core–Expression pairs are valid: of the $11 \times 9 = 99$ possible combinations, roughly thirty-five are simply absent, with no in-game explanation. Accent Sigils only modify numerical parameters; they cannot change the structure of a spell. And compatibility rules are entirely undocumented: a Vigor buff (*Focused Intent* expression) is silently cancelled when a Spectral Blur spell (also *Focused Intent*) is cast on the same target, because two spells sharing the same Expression Sigil cannot coexist—a rule the game's own AI routinely violates.

In our system, every well-typed combination is valid *by construction*, and there are no hidden compatibility rules. The type system makes all constraints explicit and statically checkable.

Ars Magica. Ars Magica [14] takes a more principled approach. Spells are specified by combining a *Technique* (one of five verbs: create, destroy, control, transform, perceive) with a *Form* (one of ten nouns representing magical domains: fire, water, mind, body, *etc.*). A fireball is Creo Ignem (create fire); mind reading is Intellego Mentem (perceive mind).

The Verb–Noun structure is a genuine product: any Technique combines with any Form, yielding fifty base combinations from which spells of any specific effect can be derived. This is closer in spirit to our approach. The difference is formalism. Ars Magica is a *tabletop* game: its system is a design vocabulary described in natural language, with no formal type discipline and no semantic guarantee. What constitutes a valid spell of a given Technique–Form combination is left to the game master's adjudication. Our system replaces that adjudication with a type checker and a denotational semantics.

2.3 Structure of the Target Type

The most novel aspect of the system is the structure of the targeting profile. Rather than encoding targeting as an opaque enum or hiding it inside a delivery-mechanism sigil (as Tyranny does), we make it an explicit product of four orthogonal dimensions.

The **allegiance** dimension is ordered:

$$\mathsf{Self} \ \leq\ \mathsf{Ally} \ \leq\ \mathsf{Any}, \qquad \mathsf{Enemy} \ \leq\ \mathsf{Any}$$

with Ally and Enemy incomparable, and $\mathsf{Ally} \vee \mathsf{Enemy} = \mathsf{Any}$. The order is not subtyping—there is no implicit coercion—but it structures the compatibility conditions. The condition "a Heal effect requires the allegiance to be at most Ally" is stated as $a \leq \mathsf{Ally}$, which is cleaner than enumerating $\{\mathit{Self}, \mathit{Ally}\}$ and must be extended whenever a new allegiance atom is added. The ordering $\mathsf{Self} \leq \mathsf{Ally}$ reflects the game-world fact that when you target your allies, you are always among them.

The **count** dimension is also ordered: $N(n) \leq \mathsf{All}$ for all n. This ordering is used in the cost function: a spell that hits all eligible targets costs more than one that hits a fixed number, and the cost function is required to be monotone.

The **geometry** dimension is a flat enumeration: Point, Line, Cone, Splash(r), or All. The geometric containment relationships (a point is a degenerate splash; a line is a degenerate cone) are mathematically natural but play no role in the type system. We note them as a possible extension and leave them aside.

Two cross-dimension compatibility conditions bear mentioning informally here. First, if the allegiance is Self, then the geometry must be Point and the count must be $N(1)$: there is only one caster, so self-targeting spells cannot be area effects. Second, when the origin is *Caster* and the allegiance is Ally, a splash geometry produces a caster-centred aura: the spell emanates from the caster and affects all allies within the radius. Allegiance always acts as a *filter* on who within the geometric area is affected.

These informal observations are the content of the compatibility side-conditions in the type system of Section 4.

3 System S: The Spell DSL

We now define System S, in which each spell carries exactly one base effect. This is the core of the paper; System M in Section 5 relaxes this restriction.

3.1 Component Types

We fix a countably infinite set BaseEffect of effect names. Particular members include *Frost*, *Fire*, *Lightning*, *Heal*, and so on; the theory is parametric in this set. We single out a subset HealingEffect $\subseteq$ BaseEffect of those effects that restore rather than damage, since they impose a compatibility condition on the target.

Duration.

$$\mathsf{DurT} \;::=\; \mathsf{Instant} \mid \mathsf{Timed}(n) \mid \mathsf{Persistent} \qquad (n : \mathbb{N},\ n \geq 1)$$

Instant completes in one game moment; Timed(n) lasts n rounds; Persistent lasts until dispelled.

Origin.

$$\mathsf{Origin} \;::=\; \mathit{Caster} \mid \mathit{Point}(p)$$

Caster anchors the spell at the caster's current position. *Point*(p) anchors it at a chosen world position p.

Allegiance. The allegiance type is the four-element partial order:

$$\mathsf{Self} \;\leq\; \mathsf{Ally} \;\leq\; \mathsf{Any}, \qquad \mathsf{Enemy} \;\leq\; \mathsf{Any}$$

with Ally and Enemy incomparable. Its Hasse diagram is shown in Fig. 1.

Geometry.

$$\mathsf{Geom} \;::=\; \mathsf{Point} \mid \mathsf{Line} \mid \mathsf{Cone} \mid \mathsf{Splash}(r) \mid \mathsf{GAll} \qquad (r : \mathbb{R}_{>0})$$

Geometry is a flat enumeration; we impose no ordering in the core system.

Count.

$$\mathsf{Count} \;::=\; N(n) \mid \mathsf{CAll} \qquad (n : \mathbb{N},\ n \geq 1)$$

We impose the partial order $N(n) \leq \mathsf{CAll}$ for all n, with distinct $N(n_1)$ and $N(n_2)$ incomparable.

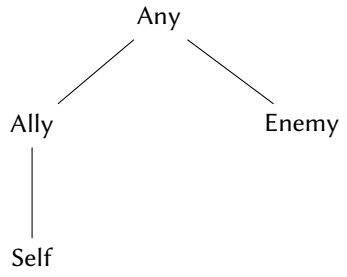


Fig. 1. Hasse diagram of the allegiance lattice.

Target type. A *targeting profile* is a four-tuple:

$$\mathsf{Target} \;::=\; \mathsf{Origin} \times \mathsf{Allegiance} \times \mathsf{Geom} \times \mathsf{Count}$$

We write a profile as (o, a, g, c).

3.2 Spell Terms

Spells are built in two phases using two operators. The operator $\otimes$ combines *distinct-dimension* components within a phase;[1] $\langle \cdot, \cdot \rangle$ then pairs the targeting profile with the spell body.

Targeting components.

$$\mathsf{tc} \;::=\; \mathsf{origin}(o) \;\mid\; \mathsf{alleg}(a) \;\mid\; \mathsf{geom}(g) \;\mid\; \mathsf{count}(c)$$

A *targeting term* T is a $\otimes$-combination of targeting components in which each dimension appears exactly once:

$$T \;::=\; \mathsf{origin}(o) \otimes \mathsf{alleg}(a) \otimes \mathsf{geom}(g) \otimes \mathsf{count}(c)$$

(The order of $\otimes$ is irrelevant; we treat it as a commutative combination.) We treat targeting terms as elements of the product type $\mathsf{Origin} \times \mathsf{Allegiance} \times \mathsf{Geom} \times \mathsf{Count}$ rather than as syntactic $\otimes$-trees. The notation $\mathsf{origin}(o) \otimes \mathsf{alleg}(a) \otimes \mathsf{geom}(g) \otimes \mathsf{count}(c)$ is sugar for the tuple (o, a, g, c). Dimension-uniqueness is thus guaranteed by construction: each component of the product must be supplied exactly once.

Body components.

$$\mathsf{bc} \;::=\; \mathsf{eff}(e) \;\mid\; \mathsf{dur}(d) \qquad (e : \mathsf{BaseEffect},\; d : \mathsf{DurT})$$

A *body term* B is:

$$B \;::=\; \mathsf{eff}(e) \otimes \mathsf{dur}(d)$$

Spell terms. A *spell term* s is a pairing of a targeting term and a body term:

$$s \;::=\; \langle T,\, B \rangle$$

[1] We use $\otimes$ as a mnemonic for combining independent pieces, not to suggest a monoidal structure. The operator is partial: $\mathsf{origin}(o_1) \otimes \mathsf{origin}(o_2)$ is undefined. As described below, targeting terms are tuples; $\otimes$ is notation for tuple construction.

3.3 Spell Types

The *spell type* of a spell is indexed by its effect:

$$\mathsf{Spell}[e] \;=\; \mathsf{Target} \times \mathsf{DurT}$$

A spell of type $\mathsf{Spell}[e]$ has effect e, some targeting profile, and some duration. The index e does not appear on the right-hand side of the definition. This is intentional: $\mathsf{Spell}[e]$ is a *phantom type* [12]. The effect tag is carried purely to enable effect-polymorphic combinators to preserve it across transformations, and to allow the Heal compatibility condition to refer to it cleanly in T-Spell. The runtime representation of $\mathsf{Spell}[e]$ is identical for all e; the index is a static discipline only. The effect index enables effect-polymorphic combinators such as:

$$\mathsf{addSplash}(r) : \mathsf{Spell}[e] \to \mathsf{Spell}[e]$$

which replaces the geometry component with $\mathsf{Splash}(r)$, regardless of the effect.

3.4 Compatibility Conditions and the Cost Function

Not every combination of component values is coherent. The type system enforces two cross-dimension *compatibility conditions* as side-conditions on the formation rule for spell terms:

(1) **Heal constraint.** If $e \in \mathsf{HealingEffect}$ then $a \leq \mathsf{Ally}$. A healing effect may target the caster or allies, but not enemies or an undiscriminated group.
(2) **Self constraint.** If $a = \mathsf{Self}$ then $g = \mathsf{Point}$ and $c = N(1)$. The caster is a single entity at a point; self-targeting spells cannot be area effects.

The two conditions are handled differently from the runtime checks in the semantics, and this asymmetry is deliberate. The Heal constraint and the Self constraint are *structural*: whether an effect is a healing effect and whether an allegiance is Self are both fixed at spell-composition time, so the type checker can verify them. By contrast, whether enemies are present in world w when the spell is cast is a property of the game world at runtime—unknowable at composition time and therefore outside the scope of the type system. The asymmetry mirrors the standard distinction in programming language theory between type-level properties (known statically) and value-level properties (known only at runtime).

The conditions are minimal by design. Every other combination is permitted; the system errs on the side of allowing unusual spells (*e.g.,* a fire bolt that damages the caster) over forbidding them.

Cost. The cost of a spell is *derived* from its type—it is not something a spell designer specifies. We define:

$$\mathsf{cost} : \mathsf{Spell}[e] \to \mathbb{N}$$

as any function satisfying the following monotonicity conditions:

- **Count monotonicity.** $c_1 \leq c_2 \;\Rightarrow\; \mathsf{cost}(\ldots, c_1, \ldots) \leq \mathsf{cost}(\ldots, c_2, \ldots)$. Hitting more targets costs at least as much.
- **Duration monotonicity.** $\mathsf{cost}(\ldots, \mathsf{Instant}, \ldots) \leq \mathsf{cost}(\ldots, \mathsf{Timed}(n), \ldots) \leq \mathsf{cost}(\ldots, \mathsf{Persistent}, \ldots)$. Longer-lasting spells cost at least as much.

No geometry monotonicity condition is imposed because Geom carries no ordering in System S (it is a flat enumeration). A natural extension would introduce $\mathsf{Point} \leq \mathsf{Splash}(r) \leq \mathsf{Splash}(r')$ for $r \leq r'$ and $\mathsf{Line} \leq \mathsf{Cone}$, and require cost to be monotone accordingly; we leave this for future work. No other constraints are imposed; the exact calibration of the cost function is a design parameter left to the implementor. A spell s of type $\mathsf{Spell}[e]$ is *castable* in a world w if $\mathsf{cost}(s) \leq \mathsf{mana}(w)$, where $\mathsf{mana}(w)$ is the caster's available mana in w. Castability is a *runtime* check; it is separate from the static type discipline.

3.5 Examples

We conclude with three well-typed spells and three ill-typed attempts.

Frost bolt. A ranged single-target instant frost attack against one enemy at position p:

$$s_1 = \langle\, \mathsf{origin}(\mathit{Point}(p)) \otimes \mathsf{alleg}(\mathsf{Enemy}) \otimes \mathsf{geom}(\mathsf{Point}) \otimes \mathsf{count}(N(1)),\ \ \mathsf{eff}(\mathit{Frost}) \otimes \mathsf{dur}(\mathsf{Instant}) \,\rangle$$

We have $s_1 : \mathsf{Spell}[\mathit{Frost}]$. Neither compatibility condition applies.

Healing aura. A caster-centred splash that heals all allies within 5 m for one minute:

$$s_2 = \langle\, \mathsf{origin}(\mathit{Caster}) \otimes \mathsf{alleg}(\mathsf{Ally}) \otimes \mathsf{geom}(\mathsf{Splash}(5)) \otimes \mathsf{count}(\mathsf{CAll}),\ \ \mathsf{eff}(\mathit{Heal}) \otimes \mathsf{dur}(\mathsf{Timed}(10)) \,\rangle$$

We have $s_2 : \mathsf{Spell}[\mathit{Heal}]$. The Heal constraint requires $a \le \mathsf{Ally}$; here $\mathsf{Ally} \le \mathsf{Ally}$. ✓

Lightning cone. A caster-origin cone of lightning hitting up to three enemies:

$$s_3 = \langle\, \mathsf{origin}(\mathit{Caster}) \otimes \mathsf{alleg}(\mathsf{Enemy}) \otimes \mathsf{geom}(\mathsf{Cone}) \otimes \mathsf{count}(N(3)),\ \ \mathsf{eff}(\mathit{Lightning}) \otimes \mathsf{dur}(\mathsf{Instant}) \,\rangle$$

We have $s_3 : \mathsf{Spell}[\mathit{Lightning}]$. No constraint applies. ✓

Ill-typed: healing enemies.

$$\langle \ldots \otimes \mathsf{alleg}(\mathsf{Enemy}) \otimes \ldots,\ \mathsf{eff}(\mathit{Heal}) \otimes \ldots \rangle \quad \text{TYPE ERROR: } \mathsf{Enemy} \not\le \mathsf{Ally}$$

Ill-typed: self-targeted splash.

$$\langle \ldots \otimes \mathsf{alleg}(\mathsf{Self}) \otimes \mathsf{geom}(\mathsf{Splash}(5)) \otimes \ldots,\ \ldots \rangle \quad \text{TYPE ERROR: } \mathsf{Self} \Rightarrow \mathsf{geom} = \mathsf{Point}$$

Ill-typed: self-targeted group.

$$\langle \ldots \otimes \mathsf{alleg}(\mathsf{Self}) \otimes \mathsf{count}(N(3)) \otimes \ldots,\ \ldots \rangle \quad \text{TYPE ERROR: } \mathsf{Self} \Rightarrow \mathsf{count} = N(1)$$

4 Type System and Semantics

This section formalises the type system for System S, gives a denotational semantics, and proves the semantic well-formedness theorem.

4.1 Typing Rules

The typing judgement has the form $\vdash s : \mathsf{Spell}[e]$. There are no typing contexts because spell terms contain no variables.

The rules are given in Fig. 2. The rule T-Target constructs a targeting term from four distinct-dimension components. The rule T-Body constructs a body term. The rule T-Spell pairs them, with the two compatibility conditions as side-conditions.

$$\frac{}{\vdash \mathsf{origin}(o) \otimes \mathsf{alleg}(a) \otimes \mathsf{geom}(g) \otimes \mathsf{count}(c) : \mathsf{Target}}\ \text{T-Target}$$

$$\frac{e \in \mathsf{BaseEffect}}{\vdash \mathsf{eff}(e) \otimes \mathsf{dur}(d) : \mathsf{BodyT}[e]}\ \text{T-Body} \qquad \frac{\begin{array}{c}\vdash T : \mathsf{Target} \quad T = (\_, a, g, c) \\ \vdash B : \mathsf{BodyT}[e] \quad B = (e, \_) \\ e \in \mathsf{HealingEffect} \ \Rightarrow\ a \le \mathsf{Ally} \\ a = \mathsf{Self} \ \Rightarrow\ g = \mathsf{Point} \ \wedge\ c = N(1)\end{array}}{\vdash \langle T, B \rangle : \mathsf{Spell}[e]}\ \text{T-Spell}$$

Fig. 2. Typing rules for System S.

The compatibility conditions appear as premises of T-Spell. They are the *only* constraints; every other component combination is permitted by the rules.

4.2 Denotational Semantics

We model the game world as an abstract set $\mathcal{W}$. A spell denotes a function from worlds to sets of successor worlds:

$$[\![s]\!] : \mathcal{W} \to \mathcal{P}(\mathcal{W})$$

The set $[\![s]\!](w)$ represents the possible outcomes of casting s in world w. Non-determinism arises naturally from area effects: if several enemies stand in a cone, the game may choose different subsets of them to be hit (for example, when the count is $N(n)$ with n less than the number of eligible targets in the area).

We interpret each component dimension separately and then combine them.

Effect denotation. Each base effect denotes a *world transformer*:

$$[\![e]\!] : \mathcal{W} \to \mathcal{P}(\mathcal{W})$$

The precise definition is effect-specific and outside the scope of this paper; we require only that $[\![e]\!](w) \neq \emptyset$ for every well-typed effect e and every world w. Informally, applying an effect always produces at least one possible outcome.

Target denotation. A targeting profile (o, a, g, c) denotes the set of entities in world w that the spell can reach:

$$[\![o, a, g, c]\!](w) \subseteq \mathsf{Entities}(w)$$

where $\mathsf{Entities}(w)$ is the set of entities in world w. Concretely:

- The origin o fixes the anchor point: *Caster* uses the caster's position; $\mathit{Point}(p)$ uses p.
- The geometry g defines the area around the anchor point.
- The allegiance a filters entities in the area: Self retains only the caster, Ally retains the caster and allies, Enemy retains enemies, Any retains all.
- The count c selects a subset of the filtered entities: $N(n)$ selects any n of them (non-deterministically); CAll selects all of them.

We require that the target denotation is non-empty whenever the compatibility conditions are satisfied. In particular:

- When $a = \mathsf{Self}$, we have $g = \mathsf{Point}$ and $c = N(1)$ (by the Self constraint), so $[\![o, \mathsf{Self}, \mathsf{Point}, N(1)]\!](w)$ is the singleton $\{\mathit{caster}(w)\}$, which is non-empty as long as the caster exists in w.
- When $a = \mathsf{Ally}$, the filtered entity set includes at least the caster (since $\mathsf{Self} \leq \mathsf{Ally}$), so $[\![o, \mathsf{Ally}, g, c]\!](w) \neq \emptyset$.
- When $a = \mathsf{Enemy}$ or $a = \mathsf{Any}$, we require, as a precondition on casting, that the relevant entities exist. This is a runtime check analogous to the castability check for mana, not a type-level constraint.

Duration denotation. Duration refines the effect denotation by selecting those successor worlds in which the effect persists for the required amount of time. Formally, $[\![e, d]\!](w, X)$ is a subset of $[\![e]\!](w)$: it retains exactly those outcome worlds that record the effect e on target set X as lasting for at least the duration d. For Instant, no persistence is required and every outcome qualifies; for $\mathsf{Timed}(n)$, the outcome world must mark the effect as lasting n rounds; for Persistent, the effect is marked as indefinite until dispelled. The precise bookkeeping is effect-specific and outside the scope of this paper. We require only:

$$[\![e, d]\!](w, X) \neq \emptyset \;\text{ whenever }\; [\![e]\!](w) \neq \emptyset \;\text{ and }\; X \neq \emptyset.$$

That is, adding a duration constraint never eliminates all outcomes of a well-formed effect.

Spell denotation. The denotation of a full spell $s = \langle T,\ B\rangle$ with $T = (o, a, g, c)$ and $B = (e, d)$ is:

$$[\![s]\!](w) \ = \bigcup_{X \in [\![o,a,g,c]\!](w)} [\![e, d]\!](w, X)$$

where $[\![e, d]\!](w, X)$ denotes the worlds resulting from applying effect e with duration d to the entity set X in world w. The union ranges over all non-deterministic target selections. Again, we require $[\![e, d]\!](w, X) \neq \emptyset$ for all non-empty X; applying a well-formed effect to a non-empty target set always produces at least one outcome.

4.3 Semantic Adequacy

The type system operates at two levels. First, it enforces structural constraints: the compatibility conditions in T-Spell are checked statically. Second, the well-definedness of the denotation rests on axioms about the base-effect vocabulary (non-emptiness of $[\![e]\!](w)$ and $[\![e, d]\!](w, X)$), which are properties of the chosen effect set, not of the type system itself. The theorem below shows that these two levels together suffice: no further conditions are required to guarantee that a well-typed spell has at least one outcome. The proof is concise precisely because the type system is minimal: the two compatibility conditions in T-Spell are exactly what is needed—no more, no less.

Theorem 4.1 (Semantic Adequacy). *If* $\vdash s : \mathsf{Spell}[e]$, *then for every world* w *in which the caster exists and in which the runtime conditions on enemy/any targets are satisfied,* $[\![s]\!](w) \neq \emptyset$.

In plain terms: every well-typed spell has at least one possible outcome when cast. The type system guarantees that there is no combination of components that produces an undefined or vacuous spell.

Proof. By induction on the typing derivation. The only rule that produces a spell type is T-Spell, so it suffices to check that rule.

Let $s = \langle T,\ B\rangle$ with $T = (o, a, g, c)$ and $B = (e, d)$. We must show $[\![s]\!](w) \neq \emptyset$, *i.e.,* there exists some outcome world.

Step 1: $[\![o, a, g, c]\!](w) \neq \emptyset$. We case-split on a:

- $a = \mathsf{Self}$: by the Self premise of T-Spell, $g = \mathsf{Point}$ and $c = N(1)$. The denotation is $\{caster(w)\}$, non-empty by assumption.
- $a = \mathsf{Ally}$: the entity set includes at least the caster, so it is non-empty. Selecting any $N(n)$ or CAll of the set preserves non-emptiness (since $n \geq 1$).
- $a = \mathsf{Enemy}$ or $a = \mathsf{Any}$: non-emptiness follows from the runtime precondition.

Step 2: For each non-empty $X \in [\![o, a, g, c]\!](w)$, we have $[\![e, d]\!](w, X) \neq \emptyset$ by the requirement on effect denotations.

Step 3: $[\![s]\!](w) = \bigcup_X [\![e, d]\!](w, X)$ is a union of at least one non-empty set, hence non-empty. □

The proof makes clear why exactly two compatibility conditions suffice. The Self constraint ensures the target denotation is always non-empty for self-targeted spells (without it, one could construct a self-targeted area spell with an empty target set). The Heal constraint ensures that a healing effect is never applied to an entity set that cannot be healed (enemies or an undiscriminated group). Every other potential constraint is either subsumed by these two or unnecessary.

The theorem separates static well-formedness from runtime resource availability. The type system guarantees structural coherence; mana sufficiency and the existence of valid enemy targets are runtime conditions checked at cast time. This mirrors the separation between type safety and resource correctness familiar from programming language theory.

5 System M: Multiple Simultaneous Effects

System S restricts each spell to a single base effect. This is clean and sufficient for most practical spells, but it rules out a natural class: spells that produce multiple distinct effects at once. Think of a *Frostfire* attack that both burns and chills its target, or a *Radiant Mending* that heals while also providing a brief shield of light. System M lifts the single-effect restriction.

5.1 Multi-Effect Spell Types

In System M, the effect component of a spell is a *non-empty set* of base effects:

$$\mathsf{Spell}[E] \;=\; \mathsf{Target} \times \mathsf{DurT} \qquad (E \in \mathcal{P}_+(\mathsf{BaseEffect}))$$

The type is now indexed by the set E rather than a single element. System S is the special case $|E| = 1$.

The body term is updated accordingly:

$$B_M \;::=\; \mathsf{effs}(E) \otimes \mathsf{dur}(d) \qquad (E \neq \emptyset)$$

5.2 Simultaneous Application

The key semantic question is: when a spell carries $\{e_1, \ldots, e_n\}$, in what order are the effects applied?

We adopt *simultaneous application*: all effects in E are applied at the same time. The denotation is the intersection of the individual effect denotations:

$$[\![E]\!](w, X) \;=\; \bigcap_{e \in E} [\![e]\!](w, X)$$

A world w' is an outcome of casting the multi-effect spell on target set X if and only if it is consistent with every individual effect being applied.

This choice deserves justification. An alternative would be an *ordered* model, in which E is replaced by a list and the effects are applied sequentially. In an ordered model, a Frostfire spell with [*Fire*, *Heal*] would differ from one with [*Heal*, *Fire*]: the first burns and then heals the damage, the second heals first and then burns. But sequential application of effects is exactly *spell-to-spell composition*, the kind of composition we explicitly set aside in Section 1. Adopting ordered effects in System M would conflate the two kinds of composition, obscuring the clean separation that is the paper's central thesis. Simultaneous application keeps the distinction sharp.

5.3 Compatibility Conditions

The compatibility conditions of System S generalise straightforwardly. The Heal constraint is lifted to *every* healing effect in the set:

(1) **Heal constraint (lifted).** $\forall\, e \in E.\ e \in \mathsf{HealingEffect} \;\Rightarrow\; a \leq \mathsf{Ally}$.
(2) **Self constraint.** Unchanged: $a = \mathsf{Self} \;\Rightarrow\; g = \mathsf{Point} \;\wedge\; c = N(1)$.

The constraints must be *jointly satisfiable*. Concretely, if E contains any healing effect, then the allegiance must be at most Ally. This means a multi-effect spell containing both *Heal* and *Fire* is well-typed only when the allegiance is Self or Ally. The spell heals its allies and burns them at the same time—unusual, but not incoherent; the type system permits it. If the allegiance is Enemy, the spell is rejected: you cannot simultaneously heal and target enemies, because healing is allegiance-constrained.

Three examples illustrate the rules:

- $E = \{\mathit{Fire}, \mathit{Heal}\}$, $a = \mathsf{Ally}$: **well-typed**. $\mathsf{Ally} \leq \mathsf{Ally}$. ✓
- $E = \{\mathit{Fire}, \mathit{Heal}\}$, $a = \mathsf{Enemy}$: **ill-typed**. $\mathit{Heal} \in \mathsf{HealingEffect}$ and $\mathsf{Enemy} \not\leq \mathsf{Ally}$. ×

- $E = \{\textit{Fire}, \textit{Lightning}\}$, a = Any: **well-typed**. Neither *Fire* nor *Lightning* is a healing effect; no constraint. ✓

5.4 Well-Formedness in System M

Theorem 4.1 extends to System M with a straightforward modification: the semantic precondition on effect denotations becomes $[\![E]\!](w, X) = \bigcap_{e \in E} [\![e]\!](w, X) \neq \emptyset$ for every non-empty X, whenever the lifted compatibility conditions hold.

This requires an additional assumption: the individual effect denotations are *jointly consistent*, meaning their intersection is non-empty. Two effects are jointly consistent when their simultaneous application does not produce a contradiction. For example, *Fire* and *Heal* both modify the health of a target; applying them simultaneously yields a world that records both the fire damage and the healing—the net change depends on their magnitudes, but the resulting world is coherent. Inconsistency would arise only if two effects imposed mutually exclusive requirements on the outcome world: for instance, an effect that demands the target is alive and a second that demands the target is dead. In a well-designed effect vocabulary such contradictions do not occur; we formalise this as an axiom on the effect denotation family.

Under this assumption, the adequacy theorem extends to System M:

Corollary 5.1 (Semantic Adequacy for System M). *If $\vdash s : \mathsf{Spell}[E]$ under the System M typing rules, then for every world w in which the caster exists, the runtime conditions on enemy/any targets are satisfied, and the effects in E are pairwise jointly consistent in w, we have $[\![s]\!](w) \neq \emptyset$.*

The proof is the same as that of Theorem 4.1, with $[\![e]\!]$ replaced by $[\![E]\!]$ and the non-emptiness axiom replaced by the joint-consistency axiom.

6 Application: D&D 5th Edition

The compositional spell system is not merely theoretical. To demonstrate its practical reach, we show how it maps onto the spell mechanics of *Dungeons & Dragons* 5th Edition (D&D 5e) [3]. The mapping is semi-formal: we give precise definitions for each concept but do not re-prove metatheorems.

6.1 Spell Slots and Level Assignment

In D&D 5e, spells occupy discrete *spell slots* at levels 1 through 9. A caster of sufficient level has a fixed number of slots at each level per day. Our cost function $\mathsf{cost} : \mathsf{Spell}[E] \to \mathbb{N}$ is continuous, so we need a translation.

Definition 6.1 (Slot level). Define $\mathsf{level} : \mathbb{N} \to \{1, \ldots, 9\}$ as any monotone surjection from cost values to D&D slot levels.

The choice of level is a design parameter fixed by calibrating the cost function against reference spells (see below). We state the definition in this generality to separate the structural property (monotonicity, which follows from cost monotonicity) from the specific breakpoints, which are a calibration detail. The choice of level corresponds to calibrating the cost function against the D&D power curve. A reasonable calibration proceeds by fixing benchmark spells. A single-target instant damage spell against one enemy (*e.g., Chromatic Orb*) is a level-1 spell; a small-area instant damage spell against multiple enemies (*e.g., Fireball*) is level 3; a persistent area effect (*e.g., Wall of Fire*) is level 4; and so on. One picks the cost weights for each component dimension so that the cost function assigns values consistent with these benchmarks, then defines level as the appropriate bucketing of the resulting range.

The key property is monotonicity: a more powerful spell (larger geometry, more targets, longer duration) has higher cost and therefore occupies a higher slot level. This is immediate from the monotonicity conditions on cost imposed in Section 3.4.

6.2 Metamagic as Component Substitution

Sorcerers in D&D 5E may spend *sorcery points* to apply *Metamagic* options that modify a spell at the moment of casting. Each Metamagic option is, in our framework, a *component substitution*—a function that replaces one component of a spell with a different value while leaving the rest intact.

Definition 6.2 (Component substitution). A *component substitution* σ on $\mathsf{Spell}[E]$ replaces the value of one component dimension with a new value, subject to the compatibility conditions:

$$\sigma : \mathsf{Spell}[E] \rightharpoonup \mathsf{Spell}[E]$$

(partial because the new component value may violate a compatibility condition, in which case σ is undefined).

The standard Metamagic options map as follows:

Twinned Spell (1 sorcery point). Replaces $\mathsf{count}(N(1))$ with $\mathsf{count}(N(2))$, targeting a second creature with the same spell. In our system: $\sigma_{twin} : \mathsf{count}(N(1)) \mapsto \mathsf{count}(N(2))$. Applicable only when the current count is $N(1)$ and the allegiance permits multiple targets.

Distant Spell (1 sorcery point). Doubles the range of a spell; for a targeted spell, replaces the origin point with a more distant one; for a self-origin spell, it is inapplicable. In our system: adjusts the origin from *Caster* to $\mathit{Point}(p')$ for a farther p', or extends the geometry radius for area spells.

Empowered Spell (1–3 sorcery points). Rerolls damage dice, effectively strengthening the effect intensity. In our system: replaces the effect e with a higher-intensity variant e^+. This requires the base-effect vocabulary to carry intensity levels, which is a natural extension.

Quickened Spell (2 sorcery points). Casts a spell as a bonus action rather than a full action. For our purposes, this corresponds to shifting the perceived temporal cost: a spell cast with Quicken is treated as $\mathsf{Instant}$ for action-economy purposes even if its duration component is $\mathsf{Timed}(n)$. In our system: a runtime reinterpretation of the duration component for scheduling purposes, not a change to the spell type.

In each case, Metamagic applies a small, well-defined modification to a single component dimension. The spell remains in the same type family $\mathsf{Spell}[E]$; the component substitution either succeeds (the new value satisfies all compatibility conditions) or it does not apply. Sorcery points serve as the additional runtime cost of the substitution.

6.3 Concentration

Some D&D 5E spells require *concentration*: the caster must maintain active focus, and at most one concentration spell can be active at any time. In our system, concentration maps directly onto the duration component.

Definition 6.3 (Concentration cost). A spell s *requires concentration* if and only if $\mathsf{dur}(s) = \mathsf{Persistent}$. We extend the castability condition with a *concentration token*: the caster holds at most one concentration token, which is consumed by any Persistent spell and released when that spell ends.

The constraint "at most one concentration spell active" is thus a global linear resource condition, analogous to the mana condition but over a Boolean resource rather than a natural-number one.

Formally, the world w carries a flag $\mathsf{conc}(w) \in \{0, 1\}$, and a Persistent spell is castable only when $\mathsf{conc}(w) = 0$.

This is a runtime condition, not a type-level constraint. The type system does not forbid a second Persistent spell from being composed; it merely cannot be *cast* while the caster is already concentrating.

6.4 Upcasting

D&D 5E allows casting a spell using a higher-level slot than the spell requires, often enhancing its effect. For example, *Cure Wounds* cast with a 2nd-level slot heals more hit points than when cast with a 1st-level slot.

In our system, upcasting corresponds to substituting a component with a more expensive one higher in the relevant partial order, funded by the slot-level surplus.

Definition 6.4 (Upcasting). Let $s : \mathsf{Spell}[E]$ with $\mathsf{level}(\mathsf{cost}(s)) = \ell$. Casting s at slot level $\ell' > \ell$ applies a component substitution σ such that $\mathsf{cost}(\sigma(s)) \leq \mathsf{slotCap}(\ell')$, where $\mathsf{slotCap}(\ell')$ is the maximum cost representable at level ℓ'. The substitution may increase count ($N(n) \to N(n')$ with $n' > n$), widen geometry ($\mathsf{Splash}(r) \to \mathsf{Splash}(r')$ with $r' > r$), or strengthen effect intensity.

Upcasting is thus a component substitution funded by extra slot expenditure. The compatibility conditions must hold after substitution; the type system guarantees the result is still well-typed.

6.5 Putting It Together

The D&D 5E machinery—spell slots, metamagic, concentration, upcasting—maps cleanly onto the component model. Spell slots are buckets of derived cost; metamagic is component substitution with a sorcery-point surcharge; concentration is a Boolean linear resource condition on Persistent spells; upcasting is surplus-funded component widening.

None of these features require changes to the core type system or the well-formedness theorem. They are interpretations of the existing component structure in the D&D 5E runtime context. A game implementing the compositional spell system could adopt whichever subset of these runtime features its design calls for.

7 Related Work

Tyranny. We have discussed *Tyranny* [13] at length in Section 2. To summarise: its sigil-based system superficially resembles component composition but is in practice a finite catalogue of pre-made spells with undocumented compatibility rules. Our work can be seen as a formal account of what Tyranny's system aspires to be.

Ars Magica. Ars Magica [14] pioneered the Technique–Form product structure for spell design. Its contribution is the insight that effects and their domains are orthogonal dimensions. Our work extends this spirit with an explicit type discipline and a formal semantics, replacing game-master adjudication with static checking.

Other compositional game systems. Divinity: Original Sin 2 [11] features rich spell-to-spell interactions: fire spells ignite oil slicks, ice creates surfaces that shock when combined with electricity. This is our composition type (2)—spell-to-spell interaction—which we deliberately leave aside. The two kinds of composition are orthogonal and could be combined in a future system.

Embedded domain-specific languages. The design philosophy of this paper draws directly on the tradition of embedded DSLs in functional programming [9]. The key idea—small combinators, orthogonal dimensions, a type system that rules out nonsense by construction—is the same principle

underlying parser combinators [10], functional reactive programming [5], and the combinator libraries surveyed by Gibbons and Wu [7]. Conal Elliott's principle of *denotational design* [4]—assign a precise mathematical meaning to library types, then derive operations as meaning-preserving functions—directly motivates our approach to spell semantics.

Compositional game theory. Hedges [8] and collaborators have developed a category-theoretic framework for game theory in which games are composed using string diagrams. Their notion of an *open game* is a game with an explicit interface to its context, enabling modular composition. While the domain is economic games rather than role-playing spells, the underlying philosophical commitment—that composition should be a first-class operation with a formal semantics—is shared.

Type systems for domain-specific languages. The use of indexed types (*e.g.*, Spell[e] indexed by the effect) to enable effect-polymorphic combinators is standard in the typed DSL literature [2]. The compatibility side-conditions in our typing rules are instances of *refinement types* [6]: the allegiance dimension carries a constraint ($\leq$ Ally for healing effects) that is a predicate on the value. A full implementation might use a refinement type checker or a dependently typed host language to automate these checks.

8 Conclusion

We have presented a compositional spell system in the functional pearl tradition. The key insight is that a spell is not an atomic incantation to be looked up in a list, but a structured value assembled from orthogonal components: an effect, a targeting profile, and a duration. A small type system with a handful of compatibility conditions—Heal targets allies, Self implies Point-and-one—is enough to guarantee that every well-typed spell has a well-defined denotation. The system is genuinely compositional: the space of valid spells grows as a product of independent component choices, not as a fixed catalogue.

System S establishes the core theory for single-effect spells. System M extends it to sets of simultaneously-applied effects, and the discussion in Section 6 shows how the component model maps cleanly onto D&D 5e's machinery of spell slots, metamagic, concentration, and upcasting.

Several directions remain open. The denotational model is deterministic; a natural extension is probabilistic effects, where $[\![s]\!](w)$ becomes a distribution over worlds rather than a set. Spell-to-spell composition—the interaction of two separately cast spells—is deliberately out of scope here, but combining the present framework with the elemental interaction systems found in games like *Divinity: Original Sin 2* is a promising avenue. Finally, the cost algebra sketched in Section 3.4 opens the door to automated balancing: given a cost model calibrated against a reference spell list, one could certify that a newly composed spell is within a given power band.

Acknowledgments

The author used Claude (Anthropic) to assist in drafting and structuring portions of this work. All content, including formal definitions, typing rules, proofs, and bibliography entries, was *not* reviewed and edited by the author, who *does not* take full responsibility for their correctness.

References

[1] [n. d.]. Tyranny Wiki: Spells. https://tyranny.wiki.fextralife.com/Spells. Community documentation; accessed 2025.

[2] Jacques Carette, Oleg Kiselyov, and Chung-Chieh Shan. 2009. Finally Tagless, Partially Evaluated: Tagless Staged Interpreters for Simpler Typed Languages. *Journal of Functional Programming* 19, 5 (2009), 509–543. doi:10.1017/S0956796809007205

[3] Jeremy Crawford, Mike Mearls, and James Wyatt. 2014. *Player's Handbook: Dungeons & Dragons, 5th Edition.* Wizards of the Coast.

[4] Conal Elliott. 2009. *Denotational Design with Type Class Morphisms (Extended Version).* Technical Report. LambdaPix. Available at http://conal.net/papers/type-class-morphisms/.
[5] Conal Elliott and Paul Hudak. 1997. Functional Reactive Animation. In *Proceedings of the 2nd ACM SIGPLAN International Conference on Functional Programming (ICFP 1997)*. 263–273. doi:10.1145/258948.258973
[6] Timothy Freeman and Frank Pfenning. 1991. Refinement Types for ML. In *Proceedings of the ACM SIGPLAN 1991 Conference on Programming Language Design and Implementation (PLDI 1991)*. 268–277. doi:10.1145/113446.113468
[7] Jeremy Gibbons and Nicolas Wu. 2014. Folding Domain-Specific Languages: Deep and Shallow Embeddings. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP 2014)*. 339–347. doi:10.1145/2628136.2628138
[8] Jules Hedges. 2018. Morphisms of Open Games. *Electronic Notes in Theoretical Computer Science* 341 (2018), 151–177. doi:10.1016/j.entcs.2018.11.008
[9] Paul Hudak. 1996. Building Domain-Specific Embedded Languages. *Comput. Surveys* 28, 4es (1996), 196. doi:10.1145/242224.242477
[10] Graham Hutton. 1992. Higher-Order Functions for Parsing. *Journal of Functional Programming* 2, 3 (1992), 323–343. doi:10.1017/S0956796800000411
[11] Larian Studios. 2017. Divinity: Original Sin 2. PC/Mac video game.
[12] Daan Leijen and Erik Meijer. 1999. Domain Specific Embedded Compilers. In *Proceedings of the 2nd Conference on Domain-Specific Languages (DSL 1999)*. 109–122. doi:10.1145/331960.331977 Introduces phantom types as a technique for static enforcement of domain constraints without runtime overhead.
[13] Obsidian Entertainment. 2016. Tyranny. PC/Mac video game.
[14] Jonathan Tweet and Mark Rein-Hagen. 2004. *Ars Magica, 5th Edition.* Atlas Games.